RBB2042 Linear Algebra & Matrix Methods

Lab Manual 2: Data Poisoning Attack and Defense

Implementing Data Poisoning Attack and Defense in Machine Learning Models

2. Learning Objectives

By the end of this lab, students will be able to:

- Understand the concept of data poisoning attacks in machine learning
- Implement a basic poisoning attack on a dataset
- Analyze its impact on model performance
- Apply simple defense mechanisms to mitigate attacks

3. Introduction

Machine learning models rely heavily on the quality of training data. Adversaries can intentionally manipulate training datasets to degrade model performance—this is known as a *data poisoning attack*. In this lab, students will simulate such attacks and apply defense techniques to enhance model robustness.

4. Tools and Requirements

- Python 3.x
- Libraries:
    - NumPy
    - Scikit-learn
    - Matplotlib (optional)
    - Pandas (optional)
- Jupyter Notebook or Python IDE

5. Dataset

Use one of the following datasets:

- Breast Cancer Dataset (from sklearn)
- MNIST dataset
- Iris dataset
- CIFAR-10 dataset

6. Procedure

Part A: Baseline Model

1. Load the dataset and split it into training and testing sets.
2. Train a classification model (Logistic Regression, SVM, or Neural Network).
3. Evaluate the model's performance.

Sample Code

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load dataset
data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.3, random_state=42
)

# Train baseline model
model = LogisticRegression(max_iter=5000)
model.fit(X_train, y_train)

# Evaluate
y_pred = model.predict(X_test)
print("Baseline Accuracy:", accuracy_score(y_test, y_pred))
```

Part B: Data Poisoning Attack

1. Modify a portion of training data using one of the following methods:
    - Label flipping
    - Feature noise injection
2. Retrain the model using poisoned data.
3. Evaluate the performance degradation.

Example: Label Flipping

```
import numpy as np

def poison_labels(y, poison_rate=0.1):
    y_poisoned = y.copy()
    n_samples = len(y)
    n_poison = int(poison_rate * n_samples)

    indices = np.random.choice(n_samples, n_poison, replace=False)
    y_poisoned[indices] = 1 - y_poisoned[indices]

     return y_poisoned
```

```
12
13  # Apply poisoning
14  y_train_poisoned = poison_labels(y_train, poison_rate=0.2)
```

1. Record the new accuracy and compare with baseline.

Part C: Defense Mechanism

1. Implement a defense strategy such as:
    - Outlier detection
    - Data cleaning
    - Robust training
2. Retrain the model using cleaned data.
3. Evaluate improvement in performance.

Example: Outlier Removal

```
1  from sklearn.ensemble import IsolationForest
2  # Defense: Isolation Forest
3  iso = IsolationForest(contamination=0.1, random state=42)
4  yhat = iso.fit_predict(X_train)

5  # Keep only "normal" samples
6  mask = yhat != -1
7  X_clean = X_train[mask]
8  y_clean = y_train_poisoned[mask]

9  # Retrain model on cleaned data
10  model.fit(X_clean, y_clean)
11 # Evaluate again
12 y_pred_clean = model.predict(X_test)
13 print("Accuracy after defense:", accuracy_score(y_test, y_pred_clean))
```

Part D: Analysis

Must analyze the following.

- Accuracy before and after poisoning
- Accuracy after applying defense techniques
- Reasons behind performance changes

Graphs and visual comparisons are required to be provided in the report.

---

7. Expected Output

- Baseline model accuracy
- Accuracy after data poisoning
- Accuracy after defense
- Comparison table or plot

---

9. Submission Requirements

For report, modify the code accordingly to fit a multiclass dataset→ **Iris dataset, using another model (SVM or neural Network)**

- Discuss briefly the differences between the two datasets and the required modification if any.
- Compare the results with different datasets, different models and varying parameters in the defense.
- Source code (Python script or notebook)
- Report (1–3 pages, PDF)
- Plots and results (screenshots or embedded figures)

Submission deadline: 25th June 2026

---

10. Discussion Questions

1. How does data poisoning affect model generalization?
2. Which defense technique was most effective and why? (refer articles on defense techniques for poisoning attack from the internet)
3. What are the limitations of the approach using the Isolation Forest?
4. What are your key observations from the lab>
5. How can real-world systems prevent such attacks?

---

11. Conclusion

This lab demonstrates the vulnerability of machine learning systems to adversarial manipulation and highlights the importance of secure data handling and robust model training techniques.
